Funções

CMA01 – Programação de Computadores I

Funções

Uma função é um trecho de código computacional que realiza uma **tarefa bem definida**.

Exemplos:

- *Calcular a área de um círculo dado seu raio;*
- *Converter graus Celsius para Fahrenheit;*
- *Calcular a nota final de um aluno dadas as notas obtidas nas avaliações;*
- *Imprimir as informações de um produto dado seu código;*
- *...*

Funções

- Por realizar uma tarefa bem definida a mesma função pode:
  - ser útil várias vezes em um mesmo programa.
  - ser útil para diferentes programas.
- A função pode receber dados para ser executada.
- A função pode produzir um resultado (por exemplo: função que calcula a área de um círculo dado seu raio).

Funções

Você já utiliza funções em seus programas:

- **`printf`** é uma função que realiza a tarefa de imprimir na tela do computador
- **`scanf`** é uma função que lê valores do teclado e armazena em variáveis indicadas entre parênteses
- **`sqrt`** é uma função que calcula e devolve o valor da raiz quadrada de um número
- **`pow`** é uma função que calcula e retorna o valor da potência, dados base e expoente indicados

Funções

Repare que você não precisa saber como as funções **`printf`**, **`scanf`**, **`sqrt`** e **`pow`** foram implementadas.

É necessário saber apenas:

- o que a função faz, isto é, qual a tarefa executada por ela (semântica);
- como ela pode ser utilizada (sintaxe).

Funções

Além de seu nome, a sintaxe da função pode (ou não) especificar:

- parâmetros – dados que a função precisa receber para executar;
- retorno – o tipo de resultado produzido pela função.

Criando e utilizando funções

Existem três situações distintas na criação e utilização de funções:

1) ***Declaração***
2) ***Definição***
3) ***Chamada***

Criando e utilizando funções

Para ilustrar a *declaração*, *definição* e *chamada* de funções, nos próximos slides teremos:

- ⇒ uma função que converte uma temperatura de graus Celsius para Fahrenheit, e
- ⇒ um programa que

(1) lê uma temperatura em graus Celsius,
(2) chama a função para convertê-la para Fahrenheit,
(3) imprime a temperatura em Fahrenheit.

Criando e utilizando funções

```
#include <stdio.h>

float celsius_fahrenheit ( float tc );

int main()
{
  float cels;
  float fahr;
  printf("Entre com temperatura em Celsius: ");
  scanf("%f", &cels);
  fahr = celsius_fahrenheit(cels);
  printf("Temperatura em Fahrenheit: ");
  printf("%f", fahr);
  return 0;
}

float celsius_fahrenheit ( float tc )
{
  float tf;
  tf = 1.8 * tc + 32;  return tf;
}
```

Declaração

Chamada

Definição

Declaração de uma função

```
#include <stdio.h>

float celsius_fahrenheit ( float tc );

int main()
{
  float cels;
  float fahr;
  printf("Entre com temperatura em Celsius: ");
  scanf("%f", &cels);
  fahr = celsius_fahrenheit(cels);
  printf("Temperatura em Fahrenheit: ");
  printf("%f", fahr);
  return 0;
}

float celsius_fahrenheit ( float tc )
{
  float tf;
  tf = 1.8 * tc + 32;
  return tf;
}
```

```
tipo_de_retorno nome_da_funcao ( lista_de_parametros );
```

A *declaração* de uma função fornece todas as informações necessárias para que o programador possa utilizá-la.

Declaração de uma função

```
#include <stdio.h>

float celsius_fahrenheit ( float tc );

int main()
{
  float cels;
  float fahr;
  printf("Entre com temperatura em Celsius: ");
  scanf("%f", &cels);
  fahr = celsius_fahrenheit(cels);
  printf("Temperatura em Fahrenheit: ");
  printf("%f", fahr);
  return 0;
}

float celsius_fahrenheit ( float tc )
{
  float tf;
  tf = 1.8 * tc + 32;
  return tf;
}
```

Toda função precisa ser *definida* ou *declarada* antes de ser usada.

Declaração de uma função

```
#include <stdio.h>

float celsius_fahrenheit ( float tc )
{
  float tf;
  tf = 1.8 * tc + 32;
  return tf;
}

int main()
{
  float cels;
  float fahr;
  printf("Entre com temperatura em Celsius:
  scanf("%f", &cels);
  fahr = celsius_fahrenheit(cels);
  printf("Temperatura em Fahrenheit: ");
  printf("%f", fahr);
  return 0;
}
```

Toda função precisa ser *definida* ou *declarada* antes de ser usada.

A *declaração* é desnecessária se a função for *definida* antes da *chamada*.

Definição de uma função

```
#include <stdio.h>

float celsius_fahrenheit ( float tc )
{
  float tf;
  tf = 1.8 * tc + 32;
  return tf;
}
```

```
tipoDeRetorno   nomeDaFuncao ( parametros )

{
   comandos
}
```

```
int main()
{
  float cels;
  float fahr;
  printf("Entre com temperatura em Celsius: ");
  scanf("%f", &cels);
  fahr = celsius_fahrenheit(cels);
  printf("Temperatura em Fahrenheit: ");
  printf("%f", fahr);
  return 0;
}
```

Definição de uma função

```
#include <stdio.h>

float celsius_fahrenheit ( float tc )
{
  float tf;
  tf = 1.8 * tc + 32;
  return tf;
}

int main()
{
  float cels;
  float fahr;
  printf("Entre com temperatura em Celsius: ");
  scanf("%f", &cels);
  fahr = celsius_fahrenheit(cels);
  printf("Temperatura em Fahrenheit: ");
  printf("%f", fahr);
  return 0;
}
```

A função pode ou não produzir um valor como resultado.

Quando um valor é produzido, seu tipo deve ser indicado.

Se não há retorno de valor, o tipo de retorno informado deve ser `void`.

Definição de uma função

```
#include <stdio.h>

float celsius_fahrenheit ( float tc )
{
  float tf;
  tf = 1.8 * tc + 32;
  return tf;
}

int main()
{
  float cels;
  float fahr;
  printf("Entre com temperatura em Celsius: ");
  scanf("%f", &cels);
  fahr = celsius_fahrenheit(cels);
  printf("Temperatura em Fahrenheit: ");
  printf("%f", fahr);
  return 0;
}
```

O *nome* da função:

- **precisa ser único**
- **deve seguir as mesmas regras de identificadores de variáveis**
- **deve identificar a ação executada pela função**

Definição de uma função

```
#include <stdio.h>

float celsius_fahrenheit ( float tc )
{
  float tf;
  tf = 1.8 * tc + 32;
  return tf;
}

int main()
{
  float cels;
  float fahr;
  printf("Entre com temperatura em Celsius: ");
  scanf("%f", &cels);
  fahr = celsius_fahrenheit(cels);
  printf("Temperatura em Fahrenheit: ");
  printf("%f", fahr);
  return 0;
}
```

A *lista de parâmetros* deve indicar os valores que precisam ser fornecidos quando a função for chamada.
Para cada valor (parâmetro), devem ser especificados:

- **tipo do parâmetro**
- **nome do parâmetro no bloco da função**

Definição de uma função

```
#include <stdio.h>

float celsius_fahrenheit ( float tc )
{
  float tf;
  tf = 1.8 * tc + 32;
  return tf;
}

int main()
{
  float cels;
  float fahr;
  printf("Entre com temperatura em Celsius: ");
  scanf("%f", &cels);
  fahr = celsius_fahrenheit(cels);
  printf("Temperatura em Fahrenheit: ");
  printf("%f", fahr);
  return 0;
}
```

O *bloco de comandos* deve ter, em seu início, a <u>declaração das variáveis</u> necessárias no código, sem incluir os identificadores da lista de parâmetros.

Só depois devem aparecer os comandos que implementam a tarefa a ser executada pela função.

Definição de uma função

```
#include <stdio.h>

float celsius_fahrenheit ( float tc )
{
  float tf;
  tf = 1.8 * tc + 32;
  return tf;
}

int main()
{
  float cels;
  float fahr;
  printf("Entre com temperatura em Celsius: ");
  scanf("%f", &cels);
  fahr = celsius_fahrenheit(cels);
  printf("Temperatura em Fahrenheit: ");
  printf("%f", fahr);
  return 0;
}
```

O *bloco de comandos* deve terminar com um <u>comando de retorno</u>, seguido do valor resultante da tarefa executada pela função.

Se a função não produzir um valor, o comando de retorno pode ser omitido.

Chamada de uma função

```
#include <stdio.h>

float celsius_fahrenheit ( float tc );

int main()
{
  float cels;
  float fahr;
  printf("Entre com temperatura em Celsius: ");
  scanf("%f", &cels);
  fahr = celsius_fahrenheit(cels);
  printf("Temperatura em Fahrenheit: ");
  printf("%f", fahr);
  return 0;
}

float celsius_fahrenheit ( float tc )
{
  float tf;
  tf = 1.8 * tc + 32;
  return tf;
}
```

```
nome_da_funcao ( lista_com_um_valor_para_cada_parametro );
```

Chamada de uma função

```
#include <stdio.h>

float celsius_fahrenheit ( float tc );

int main()
{
  float cels;
  float fahr;
  printf("Entre com temperatura em Celsius: ");
  scanf("%f", &cels);
  fahr = celsius_fahrenheit(cels);
  printf("Temperatura em Fahrenheit: ");
  printf("%f", fahr);
  return 0;
}

float celsius_fahrenheit ( float tc )
{
  float tf;
  tf = 1.8 * tc + 32;
  return tf;
}
```

A *chamada* de uma função deve incluir somente o seu *nome* e o *valor* de cada parâmetro.

O tipo de retorno e o tipo dos parâmetros *não aparecem* na chamada.

Chamada de uma função

```
#include <stdio.h>

float celsius_fahrenheit ( float tc );

int main()
{
  float cels;
  float fahr;
  printf("Entre com temperatura em Celsius: ");
  scanf("%f", &cels);
  fahr = celsius_fahrenheit(cels);
  printf("Temperatura em Fahrenheit: ");
  printf("%f", fahr);
  return 0;
}

float celsius_fahrenheit ( float tc )
{
  float tf;
  tf = 1.8 * tc + 32;
  return tf;
}
```

No caso da função `celsius_fahrenheit`, o único parâmetro, `tc`, tem tipo `float`. Então, um valor do tipo `float` precisa ser passado como parâmetro.

Na chamada, o valor armazenado pela variável `cels` é passado como parâmetro para a função.

Chamada de uma função

```
#include <stdio.h>

float celsius_fahrenheit ( float tc );

int main()
{
  float cels;
  float fahr;
  printf("Entre com temperatura em Celsius: ");
  scanf("%f", &cels);
  fahr = celsius_fahrenheit(cels);
  printf("Temperatura em Fahrenheit: ");
  printf("%f", fahr);
  return 0;
}

float celsius_fahrenheit ( float tc )
{
  float tf;
  tf = 1.8 * tc + 32;
  return tf;
}
```

As funções `printf` e `scanf` podem ser utilizadas porque a biblioteca que contém sua declaração foi incluída no início do arquivo, antes do programa.

Projetando uma função

- Ao programar, quando é identificada uma tarefa específica que pode ser codificada como uma função, é necessário primeiramente analisar a sua ***SEMÂNTICA*** para que então seja possível definir a sua ***SINTAXE***.

Projetando uma função

SEMÂNTICA: refletindo sobre a tarefa a ser executada pela função, é necessário definir:

- Quais dados de entrada são necessários?
- A tarefa deve produzir um resultado como retorno?

Projetando uma função

SINTAXE: após definir os dados de entrada e o resultado da função, é necessário especificar o tipo de cada um dos parâmetros e o tipo de retorno.

- Se a função não tem parâmetro, ainda assim os parênteses são necessários (mesmo sem conteúdo algum).
- Se não há retorno de valor, o tipo é **`void`**.

Projetando uma função

Depois de definir a declaração (ou protótipo) da função, você deve:

- identificar e declarar as variáveis adicionais necessárias;
- desenvolver uma sequência lógica de passos que realizam a tarefa atribuída à função;
- implementar estes passos traduzindo-os em instruções da linguagem C.

Ao final, deve-se usar o comando de retorno.

Execução de uma função

Com o uso de funções, a execução do programa passa a ter desvios...

Execução de uma função

```
1  #include <stdio.h>
2
3  float celsius_fahrenheit ( float tc );
4
5  int main() {
6   float cels;
7   float fahr;
8
9   printf("Entre com temperatura Celsius: " );
10  scanf("%f", &cels);
11
12  fahr = celsius_fahrenheit(cels);
13
14  printf("Temperatura em Fahrenheit: " );
15  printf("%f", fahr);
16
17  return 0;
18 }
19
20 float celsius_fahrenheit ( float tc ) {
21  float tf;
22  tf = 1.8 * tc + 32;
23  return tf;
24 }
```

Função sem retorno de valor

- Função que não devolve valor para o programa ou função que a chamou.
- Em algumas linguagens, funções com esta característica são denominadas procedimentos.
- Um procedimento normalmente executa uma ação que não gera dados.

Função sem retorno de valor

```
#include <stdio.h>

void imprimeFormatoHora (int qtdMinutos)
{
    int hora, min;
    hora = qtdMinutos / 60;
    min = qtdMinutos % 60;
    printf("%02d:%02d",hora,min);
    return;
}

int main()
{
    int minutos;
    printf("Entre com os minutos para converter em
hora:");
    scanf("%d",&minutos);
    printf("%d minutos equivale a ", minutos);
    imprimeFormatoHora (minutos);
    return 0;
}
```

Tipo de retorno

Como não há retorno de valor algum, o tipo indicado é **`void`**.

Função sem retorno de valor

```
#include <stdio.h>

void imprimeFormatoHora (int qtdMinutos)
{
    int hora, min;
    hora = qtdMinutos / 60;
    min = qtdMinutos % 60;
    printf ("%02d:%02d",hora,min);
    return;
}

int main()
{
     int minutos;
     printf ("Entre com os minutos para converter em
     hora:");
     scanf ("%d",&minutos);
     printf ("%d minutos equivale a ", minutos);
     imprimeFormatoHora (minutos);
     return 0;
 }
```

Comando de retorno

Como não há retorno de valor algum, o comando **`return`** aparece **sozinho** (apenas seguido do ponto e vírgula) **ou é omitido**.

Função sem retorno de valor

```
#include <stdio.h>

void imprimeFormatoHora (int qtdMinutos)
{
    int hora, min;
    hora = qtdMinutos / 60;
    min = qtdMinutos % 60;
    printf ("%02d:%02d",hora,min);
    return;
    printf ("Esta linha de comando nunca sera executada! " );
}

int main()
{
    int minutos;
    printf ("Entre com os minutos para converter em
    hora:");
    scanf ("%d",&minutos);
    printf ("%d minutos equivale a ", minutos);
    imprimeFormatoHora (minutos);
    return 0;
}
```

Cuidado ao usar o comando **`return`** no meio de uma função, pois, mesmo que haja comandos depois, a função é encerrada assim que o comando **`return`** for executado.

Função sem retorno de valor

```
#include <stdio.h>
void imprimeFormatoHora (int qtdMinutos)
{

    int hora, min;
    hora = qtdMinutos / 60;
    min = qtdMinutos % 60;
    printf("%02d:%02d",hora,min);
    return;
 }

int main()
{
    int minutos;
    printf("Entre com os minutos para converter em
    hora:");
    scanf("%d",&minutos);
    printf("%d minutos equivale a ", minutos);
    imprimeFormatoHora(minutos);
    return 0;
 }
```

Função com retorno de valor

```
 #include <stdio.h>

int converteEmMinutos(int numHoras, int numMinutos)
{
    int totalMinutos;
    totalMinutos = numHoras * 60 + numMinutos;
    return totalMinutos;
}

int main()
{
    int hora,min,qtdMinutos;
    printf("Entre com horas e minutos para conversao:");
    scanf("%d %d",&hora,&min);
    qtdMinutos = converteEmMinutos(hora,min);
    printf("%02d:%02d equivale a %d minutos.",hora,min,qtdMinutos);
    return 0;
}
```

Função com retorno de valor

```
#include <stdio.h>

int converteEmMinutos (int numHoras, int numMinutos)
{
    int totalMinutos;
    totalMinutos = numHoras * 60 + numMinutos;
    return totalMinutos;
}

int main()
{
    int hora,min,qtdMinutos;
    printf("Entre com horas e minutos para
    conversao:");
    scanf("%d %d",&hora,&min);
    qtdMinutos = converteEmMinutos(hora,min);
    printf("%02d:%02d equivale a %d minutos." ,
    hora, min, qtdMinutos);
    return 0;
}
```

Tipo de retorno O tipo informado deve ser definido com base no dado produzido pela função. Neste caso, o valor que a função vai retornar deve ser um número inteiro.

Função com retorno de valor

```
#include <stdio.h>

int converteEmMinutos (int numHoras, int numMinutos)
{
    int totalMinutos;
    totalMinutos = numHoras * 60 + numMinutos;
    return totalMinutos;
}

int main()
{
    int hora,min,qtdMinutos;
    printf("Entre com horas e minutos para
    conversao:");
    scanf("%d %d",&hora,&min);
    qtdMinutos = converteEmMinutos (hora,min);
    printf("%02d:%02d equivale a %d minutos." ,
    hora, min, qtdMinutos);
    return 0;
}
```

Comando de retorno

O valor que será informado no retorno da função precisa ser indicado junto ao comando **`return`**. Neste caso, este valor corresponde ao conteúdo da variável `totalMinutos`.

Função com retorno de valor

```
#include <stdio.h>

int converteEmMinutos (int numHoras, int numMinutos)
{
    int totalMinutos;
    totalMinutos = numHoras * 60 + numMinutos;
    return totalMinutos;
}

int main()
{

    int hora,min,qtdMinutos;
    printf("Entre com horas e minutos para
    conversao:");
    scanf("%d %d",&hora,&min);
    qtdMinutos = converteEmMinutos(hora,min);
    printf("%02d:%02d equivale a %d minutos." ,
    hora, min, qtdMinutos);
    return 0;
}
```

Função com retorno de valor

```
#include <stdio.h>

int converteEmMinutos(int numHoras, int numMinutos)
{
    int totalMinutos;
    totalMinutos = numHoras * 60 + numMinutos;
    return totalMinutos;
}

int main()
{

    int hora,min;
    printf("Entre com horas e minutos para conversao:" );
    scanf("%d %d",&hora,&min);
    printf("%02d:%02d equivale a %d minutos." ,hora, min,
    converteEmMinutos(hora,min));

    return 0;
}
```

Vantagem do uso de funções

Uma mesma tarefa pode ser implementada uma única vez e ser utilizada **várias vezes** (por um ou mais programas):

- erros precisam ser corrigidos em um único lugar;
- a reutilização da função (chamada) pode ser feita de forma simples;
- o código fica legível, mais fácil de ser entendido e mais compacto;
- o uso de funções possibilita a modularização do código de um programa, isto é, o desenvolvimento do código organizado em módulos funcionais.

Variáveis e escopo

Escopo: contexto que define a visibilidade e acessibilidade das variáveis em diferentes partes do programa.

Toda variável tem um escopo. O escopo da variável equivale às linhas de código onde a variável pode ser acessada, lida e/ou modificada.

Variáveis e escopo

Escopo: contexto que define a visibilidade e acessibilidade das variáveis em diferentes partes do programa.

Toda variável tem um escopo. O escopo da variável equivale às linhas de código onde a variável pode ser acessada, lida e/ou modificada.

Variáveis e escopo

São denominadas <u>variáveis locais</u>:

- as variáveis declaradas na função;
- todos os parâmetros recebidos pela função.

O escopo de uma variável local corresponde apenas ao bloco de comandos de sua função.

Dentro de uma função não se tem acesso a variáveis declaradas em outra função.

Variáveis e escopo

A reserva de memória para uma variável local é condicionada à execução da função:

- A reserva é refeita cada vez que a função é executada (podem ser reservados endereços distintos a cada execução).
- Quando a execução da função termina, os espaços de memória reservados (e as respectivas variáveis associadas) são liberados para outros usos e já não podem ser acessados.

Variáveis e escopo

Os nomes das variáveis locais coincidentemente são iguais mas elas são completamente independentes.

O endereço de memória da variável `raio` da função `main` é diferente do endereço de memória do parâmetro `raio` da função `volume_cilindro`.

```
#include <stdio.h>

#define PI 3.14159

float volume_cilindro (float raio, float altura)
{
  float volume = PI * raio * raio * altura;
  return volume;
}

int main()
{
  float raio, altura, volume;
  printf("Entre com o valor do raio: ");
  scanf("%f", &raio);
  printf("Entre com o valor da altura: ");
  scanf("%f", &altura);

  volume = volume_cilindro(raio, altura);

  printf("Volume do cilindro = ");
  printf("%f", volume);
  return 0;
}
```

Variáveis e escopo

A modificação do nome dos parâmetros e variáveis da função não implica em qualquer modificação no programa.

```
#include <stdio.h>

#define PI 3.14159

float volume_cilindro (float r, float a)
{
  float v = PI * r * r * a;
  return v;
}

int main()
{
  float raio, altura, volume;
  printf("Entre com o valor do raio: ");
  scanf("%f", &raio);
  printf("Entre com o valor da altura: ");
  scanf("%f", &altura);

  volume = volume_cilindro(raio, altura);

  printf("Volume do cilindro = ");
  printf("%f", volume);
  return 0;
}
```

Variáveis e escopo

A modificação do nome dos parâmetros e variáveis da função não implica em qualquer modificação no programa.

```
#include <stdio.h>

#define PI 3.14159

float volume_cilindro (float raio, float altura)
{
  float volume = PI * raio * raio * altura;
  return volume;
}

int main()
{
  float raio, altura, volume;
  printf("Entre com o valor do raio: ");
  scanf("%f", &raio);
  printf("Entre com o valor da altura: ");
  scanf("%f", &altura);

  volume = volume_cilindro(raio, altura);

  printf("Volume do cilindro = ");
  printf("%f", volume);
  return 0;
}
```

Variáveis e escopo

Neste exemplo, quando a função é chamada, o valor das variáveis `raio` e `altura` declaradas na função main são utilizados para inicializar os parâmetros `raio` e `altura` da função volume_cilindro.

```
#include <stdio.h>

#define PI 3.14159

float volume_cilindro(float raio, float altura)
{
  float volume = PI * raio * raio * altura;
  return volume;
}

int main()
{
  float raio, altura, volume;  printf("Entre
  com o valor do raio: ");  scanf("%f", &raio);
  printf("Entre com o valor da altura: ");
  scanf("%f", &altura);

  volume = volume_cilindro(raio, altura);

  printf("Volume do cilindro = ");
  printf("%f", volume);
  return 0;
}
```

Variáveis e escopo

- Existem alguns tipos de variáveis que não serão abordadas neste curso:
  - Variável Global:
    - declarada fora das funções;
    - vive ao longo de toda execução do programa;
    - visível por todas as funções subsequentes.
  - Variável Estática:
    - existe durante toda a execução do programa;
    - só é visível dentro da função que a declara.

Exercícios

1. Considerando a fórmula para o cálculo da distância entre dois pontos (x1, y1) e (x2, y2):

a) Escreva uma função que receba como parâmetros as coordenadas de dois pontos e retorne a distância entre eles.

b) Escreva um programa em C (função principal) que capture do teclado as coordenadas dos 3 vértices de um triângulo, calcule e imprima o perímetro deste triângulo, chamando a função anterior.

c) Teste seu programa, simulando sua execução com as seguintes coordenadas: (4,1), (1,1), (4,5).

Exercícios

2.a) Considerando Pi = 3,14159, para cada opção abaixo, escreva uma função que recebe como parâmetro o raio de um círculo e:

- retorne seu diâmetro;
- retorne sua circunferência;
- retorne sua área;
- imprima o diâmetro, a circunferência e a área chamando as funções anteriores.

b) Elabore um programa que leia do teclado o valor do raio de dois círculos e, para cada círculo, chame a função que imprime as informações.

c) Teste seu programa com os valores 1 e 3.

Exercícios

DESAFIO: Faça um programa para calcular quantas latas de verniz serão necessárias para cobrir um deque de madeira. O usuário do programa informará a largura e o comprimento da superfície a ser coberta e o programa deverá imprimir o número de latas necessárias (valor inteiro), dado que cada lata de verniz cobre até 3 m^2 de superfície. O programa deverá ter no mínimo 3 funções. Teste o programa calculando o necessário para cobrir uma superfície de 4.5 x 5m.

Observação: tente identificar as tarefas que poderão constituir diferentes funções e, para cada tarefa, especifique os dados de entrada (parâmetros) necessários para sua execução e defina se esta tarefa produzirá ou não um resultado (retorno).

Funções

CMA02 - Laboratório de Programação de Computadores I